

1. Logika cyfrowa

Układ	Formuła / sygnatura	Opis
Sumatory		
Półsumator	$s = x \oplus y$ $c = x \& y$	Dodaje dwa bity. s to suma, c to bit przeniesienia (carry).
Pełny sumator (FA)	$s = x \oplus y \oplus ci$ $co = x \& y \mid ci \& (x \oplus y)$	Jak półsumator, ale przyjmuje też przeniesienie wejściowe ci .
Sumator n-bitowy	$n \times \text{FA}$	Kaskada (ripple-carry): wyjście C_i trafia na wejście kolejnego FA. Ostatnie co to Carry Out całości.

Układy kombinacyjne		
Dekoder	$n \rightarrow 2^n$	Wejście koduje liczbę k . Na wyjściu zapalony jest tylko bit nr k .
Multiplekser	$2^n + n \rightarrow 1$	2^n bitów danych + n bitów sterujących S . Na wyjście przechodzi S -ty bit danych.
ALU	$A, B, f \rightarrow C$	Dekoder na bitach f wybiera operację (np. $A + B$, $A \mid B$, $A \& B$); multipleksowanie wyników bramkami AND/OR.

Układy sekwencyjne (pamięć)		
Przerzutnik S-R	S - set, R - reset	Przechowuje 1 bit (Q). S=1 ustawia $Q = 1$, R=1 zeruje, S=R=0 trzyma stan. Zbudowany z dwóch sprzężonych NOR -ów.
Sterowany poziomem	aktywny gdy CLK = 1	Wejścia S / R bramkowane AND-em z zegarem. Stan zmienia się tylko przy CLK=1 .
Sterowany zboczem	zbocze 1 $\rightarrow$ 0	Dwa przerzutniki poziomowe w kaskadzie (master-slave), drugi z zanegowanym zegarem. Stan przepisuje się w momencie zmiany zegara.

2. Typy danych

Typ	char	short	int	long	float	double	void*
x86-64	1	2	4	8	4	8	8

3. Kodowanie liczb i konwersja

$$\text{B2U}(X) = \sum_{i=0}^{w-1} x_i 2^i \quad \text{B2T}(X) = -x_{w-1} 2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i$$

$$\text{T2U}(x) = x < 0 ? x + 2^w : x \quad \text{U2T}(u) = u > \text{TMax} ? u - 2^w : u$$

Skróty: **B** = bity, **U** = unsigned, **T** = ze znakiem (kod U2).
Stąd **B2U** = bity $\rightarrow$ unsigned, podobnie **U2T**, **UMax**, **TMax**, **UAdd**, **TAdd**.

Stała	Bity	Wartość
UMax	11...1	$2^w - 1$
TMax	01...1	$2^{w-1} - 1$ (INT_MAX)
TMin	10...0	-2^{w-1} (INT_MIN)
−1	11...1	te same bity co UMax

Promocja w C: **unsigned** i **int** w jednym wyrażeniu/porównaniu ($< > ==$) $\rightarrow$ **int** promowany do **unsigned** (bity bez zmian, $-1 \rightarrow \text{UMax}$).

4. Rozszerzanie, obcinanie i przesunięcia

Rozszerz. 0 (zext)	unsigned : dopisuje 0 z góry
Rozszerz. znak (sext)	signed : powiela bit znaku (MSB)
x << k	$x \cdot 2^k$ (sgn./uns.)
u >> k	$\lfloor u/2^k \rfloor$ — logiczne (zera)
x >> k	arytmetyczne (kopia MSB), zaokr. do $-\infty$
$x/2^k$ do 0	(x + (1<<k)-1) >> k

Strength red.: **x*24** $\rightarrow$ **(x<<5)-(x<<3)**.

5. Operacje, overflow i endianness

UAdd/TAdd	$(u + v) \bmod 2^w$
Nadmiar (+)	$u, v > 0$, a wynik < 0 (wynik -2^w)
Nadmiar (-)	$u, v < 0$, a wynik ≥ 0 (wynik $+2^w$)
Negacja U2	-x = ~x+1 ; -TMin=TMin , -0=0
Wykr. nadmiaru (s=x+y)	((s^x)&(s^y))>>(w-1)
Maska / abs	m=x>>31; abs=(x^m)-m

Schemat	Najniższy adres	0x0123
Big Endian	MSB	[01][23]
Little Endian	LSB	[23][01]

6. Zmiennoprzecinkowe (IEEE 754)

$$V = (-1)^s \times M \times 2^E \quad \text{Bias} = 2^{k-1} - 1$$

Format	bity	s	exp (<i>k</i>)	frac (<i>n</i>)
float	32	1	8	23
double	64	1	11	52

Kategorie wartości

Kategoria	exp	Własności
Znormalizowane	$\neq 0 \dots 0$ $\neq 1 \dots 1$	$E = \text{exp} - \text{Bias}$. $M = 1.\text{frac}$. Najgęściej ułożone wokół 0.
Zdenormaliz.	$= 0 \dots 0$	$E = 1 - \text{Bias}$. $M = 0.\text{frac}$. Reprezentują +0.0 i -0.0 (frac = 0) oraz liczby bardzo bliskie zera.
Specjalne	$= 1 \dots 1$	Gdy $\text{frac} = 0$, to $+\infty$ lub $-\infty$ (nadmiar/dzielenie przez 0). Gdy $\text{frac} \neq 0$, to NaN (np. $\sqrt{-1}$, $\infty - \infty$).

Zaokrąglanie GRS i Round-to-Even

Domyślny tryb to **Round-to-Even** (zapobiega błędowi statycznemu przy sumowaniu).

... x x G

!

R S S S ...

G - ostatni zachowany, R - pierwszy usunięty, S - OR reszty

Warunek	Ułamek	Akcja
R=0	< 0.5	W dół (odcięcie)
R=1, S=1	> 0.5	W górę (+1 do G)
R=1, S=0	= 0.5	Do parzystej: w górę gdy G=1, w dół gdy G=0

Własności matematyczne i rzutowanie

- **Brak łączności:** $(a + b) + c \neq a + (b + c)$, przez zaokrąglenia i utratę danych.
- **Brak rozdzielności:** $a(b + c) \neq ab + ac$.
- **Rzutowanie int → float:** Może stracić precyzję (zaokrągła zgodnie z trybem).
- **Rzutowanie int → double:** Dokładne i bezstratne (bo 52 bity mantysy > 32 bity inta).
- **Rzutowanie float/double → int:** Obcina ułamek w stronę 0. Jeśli poza zakresem lub NaN → z reguły zwraca TMin (0x80000000).

7. Rejestry x86_64

%rax	%eax	%ax %ah %al	%rbx	%ebx	%bx %bh %bl
%rcx	%ecx	%cx %ch %cl	%rdx	%edx	%dx %dh %dl
%rsi	%esi	%si %sil	%rdi	%edi	%di %dil
%rbp	%ebp	%bp %bpl	%rsp	%esp	%sp %spl
%r8	%r8d	%r8w %r8b	%r9	%r9d	%r9w %r9b

Uwaga: Rejestry od %r10 do %r15 używają schematu:

%r10, %r10d, %r10w, %r10b.

Podział ról rejestrów

%rax	Akumulator. Główny rejestr arytmetyczny. Przechowuje wartość zwracaną .
%rdi, %rsi, %rdx, %rcx, %r8, %r9	Kolejno: od 1. do 6. argumentu funkcji . Caller-saved.
%rsp	Wskaźnik stosu (SP). Wskazuje wierzchołek ramki.
%rbp	Wskaźnik bazy (BP). Początek ramki stosu. Callee-saved.
%rbx, %r12-%r15	Ogólnego przeznaczenia. Wszystkie callee-saved .
%rip	Wskaźnik instrukcji (Program Counter).

8. Tryby adresowania (operandsy)

Tryb	Format	Obliczanie adresu
Natychmiastowy (Imm)	\$Imm	Imm
Rejestrowy (Reg)	%Ra	%Ra
Bezpośredni (Mem)	Imm	M[Imm]
Pośredni (Mem)	C(%Rb)	M[C(%Rb)]
Z przesunięciem	D(%Rb)	M[C(%Rb) + D]
Skalowany (Ind/scaled)	D(%Rb, %Ri, S)	M[C(%Rb + %Ri * S + D)]
Skalowany (baseless)	(, %Ri, S)	M[%Ri * S]

Legenda: Imm / D to stała liczbowa, %Ra / %Rb to rejestr bazowy, %Ri to rejestr indeksowy, a S to skala (tylko: 1, 2, 4 lub 8).

9. Flagi stanu i sterowanie

ZF	Zero. Wynik to 0 (np. argumenty są równe).	
SF	Sign. Wynik jest ujemny (MSB = 1).	
CF	Carry. Przepełnienie dla liczb bez znaku (unsigned).	
OF	Overflow. Przepełnienie dla liczb ze znakiem (signed).	
cmp S1, S2	S2 − S1	Ustawia flagi jak odejmowanie, nie zapisuje wyniku.
test S1, S2	S2 & S1	Ustawia flagi jak AND (np. test czy rejestr jest zerem).
jX dest	if(X) %rip ← dest	Skok warunkowy (X = warunek).
setX D	if(X) D ← 1	Ustawia bajt (np. %al) na 0 lub 1 na podstawie flag.
cmovX S, D	if(X) D ← S	Warunkowe kopiowanie (optymalizacja zamiast skoku).

Sufiksy warunkowe (dla jX, setX, cmovX)

Sufiks	Synonim	Znaczenie
e / z		Equal / Zero (równe / wynik to 0)
ne / nz		Not Equal / Not Zero (nierówne)
s		Sign (wynik ujemny, SF=1)

Liczby ze znakiem (signed)		
g / ge		Greater / Greater or Equal
l / le		Less / Less or Equal

Liczby bez znaku (unsigned)		
a / ae	nc	Above / Above or Equal (No Carry)
b / be	c	Below / Below or Equal (Carry)

Translacja struktur kontrolnych (C → ASM)

- if-else:** Realizowane przez skoki (jX) lub cmovX. cmovX unika kar za błędną predykcję skoku, ale oblicza zawsze obie ścieżki. Nie używa się go, gdy operacje są kosztowne obliczeniowo lub mają efekty uboczne (np. x++).
- Pętla do-while:** Ciało pętli jest na początku, na końcu znajduje się test i warunkowy skok w górę (if(war) goto loop).
- Pętla while:**
 - Jump-to-middle (-Og):** początkowy skok omija ciało pętli prosto do testu na końcu.
 - Do-while conversion (-O1):** początkowy strażnik (if(!war) goto done), po którym następuje zwykła pętla typu do-while.
- Pętla for:** Kompilator konwertuje je na pętle while według schematu: init; while(war){ body; update it }.
- Switch:** Używa tablic skoków dla osiągnięcia czasu skoku O(1) przy case. Realizowane przez pośredni skok z adresowaniem skalowanym: jmp *.L4(%rdi,8). Pominięcie break w języku C odpowiada fizycznemu brakowi instrukcji skoku kończącej dany blok w ASM (fall-through do kodu poniżej).

10. Katalog instrukcji (AT&T)

Opcode	Efekt	Opis
mov S, D	D ← S	Kopiuje wartość z S do D.
lea S, D	D ← addr(S)	Oblicza adres S (bez czytania pamięci).
add S, D	D ← D + S	Dodawanie (ustawia flagi).
sub S, D	D ← D − S	Odejmowanie (ustawia flagi).
imul S, D	D ← D * S	Mnożenie liczb ze znakiem.
sal / shl k, D	D <=< k	Przesunięcie w lewo (mnożenie przez 2 ^k).
sar k, D	D >=> k	Arytmetyczne w prawo (dzielenie). Kopiuje znak.
shr k, D	D >=> k	Logiczne w prawo. Dopełnia zerami.
and / or S, D	D ← D & / S	Operacje bitowe (ustawiają flagi).
xor S, D	D ← D ^ S	Często używane jako xor %rax, %rax do zerowania.

Porównania i sterowanie

cmp S1, S2	S2 − S1	Ustawia flagi jak odejmowanie, nie zapisuje wyniku.
test S1, S2	S2 & S1	Ustawia flagi jak AND .
jX dest	if(X) %rip ← dest	Skok warunkowy (X = warunek).
setX D	if(X) D ← 1	Ustawia najmłodszy bajt na 0 lub 1 na podstawie flag.
cmovX S, D	if(X) D ← S	Warunkowe kopiowanie (optymalizacja zamiast skoku).

Uwaga o sufiksach: Instrukcje przyjmują sufiks określający rozmiar danych: b (8-bit), w (16-bit), l (32-bit), q (64-bit).
Np. movl kopiuje 32 bity (i automatycznie zeruje górną połowę 64-bitowego rejestru!).